GraphMind Support

A multilingual voice support agent whose memory is a FalkorDB graph. It remembers every customer, every fix, and how it reached each answer.

Live: **graphmind-support.onrender.com** · Code: **github.com/ramrajsingh/graphmind_support**

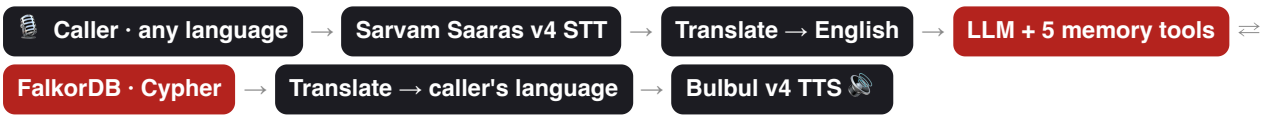
FalkorDB: primary memory · Track 02 · Agent Memory · Track 01 · Connected Data · Vector + multi-hop Cypher · Sarvam AI · 22 Indian languages · Explainable: Cypher per turn

The problem

A customer calls back: “My account still isn't working.” Most support bots start from zero. They ask the customer to explain again, suggest the fix that already failed, and forget that the customer spoke Hindi yesterday and English today. The answer depends on **how things relate**: this customer, this ticket, the steps already tried, and the fixes that worked for others with the same problem.

What we built

Asha is a voice agent that uses FalkorDB as its context layer. On every call she loads the customer's subgraph: products, tickets, steps tried, and summaries of earlier calls. She matches new problems to known issues with vector search, and gets the next fix with a multi-hop traversal: *known issue → ranked fixes, minus what this ticket already tried*. When a fix works, she writes it back as shared knowledge, so the next customer benefits in any language.



Memory model in FalkorDB

Memory	Graph	What it's for

Semantic shared, learned	`(:KnownIssue)-[:RESOLVED_BY {rank}]->(:TroubleshootingStep)`
	`(:KnownIssue)-[:AFFECTS]->(:Product)`
	vector index on `KnownIssue.embedding`
		Match a new problem to a known issue (cosine distance < 0.5) and its proven fixes, in order

Customer state per customer	`(:Customer)-[:OPENED]->(:Ticket)-[:MATCHES]->(:KnownIssue)` `(:Ticket)-[:TRIED | RESOLVED_WITH]->(:TroubleshootingStep)`	What was tried, what's left, contacts and priority
Episodic	`(:Customer)-[:HAD_CALL]->(:Conversation {summary})-[:HAS_MESSAGE]->(:Message)` `(:Conversation)-[:DISCUSSED]->(:Ticket)`	Each call's messages, original language and an LLM summary, read back at the next call
Procedural	Agent playbook	One step at a time, never repeat a `TRIED` step, escalate when no fixes are left, raise priority on repeat contact

Multi-hop: the next fix for a callback

```
MATCH (c:Customer
{customer_id:$cid})-[:OPENED]->
(t:Ticket)
OPTIONAL MATCH (t)-[:MATCHES]->
(k:KnownIssue)
```

Vector search + traversal: a new problem

```
CALL
db.idx.vector.queryNodes('KnownIssue','embedding',6,
     vecf32($vec)) YIELD node AS k, score
WITH k, score WHERE score < 0.5
OPTIONAL MATCH (k)-[:AFFECTS]->(p:Product)
```

```
OPTIONAL MATCH (t)-[:TRIED]->
(f:TroubleshootingStep)
WITH t, k, collect(f.name) AS tried
OPTIONAL MATCH (k)-
[rb:RESOLVED_BY]->(s)
WHERE NOT (t)-[:TRIED]->(s)
WITH t, k, tried, s ORDER BY
rb.rank
RETURN t.id, k.name, tried,
collect(s.name) AS untried
```

Preloaded at call start, so a callback is answered with **no extra tool round trip** (31 ms query).

```
WITH k, score, p WHERE p.key = $product
MATCH (k)-[rb:RESOLVED_BY]->(s:TroubleshootingStep)
OPTIONAL MATCH (:Ticket {id:$tid})-[tr:TRIED]->(s)
RETURN k.name, score,
  collect({step:s.name, tried: count(tr) > 0})
```

Unrelated problems (“router LED blinking orange”) correctly match nothing, which triggers learning.

Learning loop: memory that improves

When a caller confirms a fix worked, Asha adds a `RESOLVED_WITH` link on the ticket and promotes the fix to shared knowledge: a `RESOLVED_BY {learned:true}` link on the matched known issue, or a **new embedded KnownIssue** created from the ticket. Tested live:

1. **Rahul (Hindi)**: “Mera router ka light orange blink kar raha hai.” No known issue matches. A ticket is opened. A factory reset fixes it, so a new KnownIssue is created with that fix.
2. **Ananya (Bengali)**: “আমার Acme Smart Router এর আলো কমলা রঙে জ্বলছে-নিভছে” (different words, different language). Vector search finds Rahul's learned issue, and Asha suggests the factory reset first.

Explainable by design

Every turn returns the exact Cypher it ran and how long each query took. The UI shows them under *Agent memory calls & Cypher*, next to a live graph view where the nodes and edges used in the answer are highlighted. Judges can see why Asha suggested a step: which known issue matched, which steps were excluded as `TRIED`, and in what rank order the rest came.

Sarvam multilingual layer

Hear and understand

Saaras v4 transcribes 22 Indian languages plus English, including mixed speech like Hinglish, with language detection and keyterms (“Acme Pay”, “2FA”). **Sarvam Translate** turns each turn into English, so **one language-neutral graph** serves every language: a ticket opened in Hindi is continued in Tamil or English.

Speak naturally

Replies are translated back in classic-colloquial style, keeping tech terms in English, and spoken by **Bulbul v4** with a persona voice per language. Typed caller lines are read aloud in a male or female caller voice, so a typed demo sounds like a two-person call. `preferred_language` on the Customer node sets the greeting language.

Live demo transcript (real run)

Call 1: Rahul, Hindi

Mera Acme Pay app update ke baad login nahi ho raha hai.
“My Acme Pay app won't log in since the update”

🔧 `start_new_issue`: vector match *Login fails after app update*, ticket opened (85 ms + 29 ms)
Rahul, चलिए Acme Pay का app cache clear करते हैं…
“Let's start by clearing the app cache…”

Cache clear kar diya, phir bhi login nahi ho raha.
→ 🔧 record_step: `(:Ticket)-[:TRIED]->(Clear app cache)`

App bhi update kar liya, abhi bhi nahi chal raha. Main baad mein call karta hoon.
→ TRIED (Update app) · hang up → call summary saved on the `:Conversation` node

Call 2: next day, English

Hi, my account still isn't working.

🔧 `update_ticket(repeat_contact)`: contacts 2, priority **high**
“Let's try re-linking your 2FA device…”
The untried fix came from the preloaded traversal. No re-explaining and no repeated steps.

Yes, I re-linked the 2FA device and now I can log in. Thank you!

→ RESOLVED_WITH · ticket resolved · fix confirmed in shared memory

Performance (FalkorDB Cloud)

28–160 ms	~2.3 s	~5 s	0
per Cypher query	agent step on a callback	full callback turn, speech in → speech out	repeated steps across calls

Speedups: the customer subgraph is preloaded in one traversal, new issues use one combined tool, independent tool calls run in parallel, messages are logged in the background, and Bulbul synthesizes each sentence in parallel.

Stack and setup

FalkorDB (Cloud or Docker; vector, full-text and range indexes) · **Sarvam AI** (Saaras v4, Translate, Bulbul v4) · **OpenAI GPT-4.1** (or Claude) with tool use · text-embedding-3-small · FastAPI · vis-network UI.

```
uv venv && uv pip install -r requirements.txt
cp .env.example .env                    # FalkorDB, Sarvam, OpenAI keys
python -m scripts.setup_graph           # indexes + seed graph
uvicorn app.server:app --port 8000      # open http://localhost:8000
```

Live demo: https://graphmind-support.onrender.com · **Repo:** github.com/ramrajsingh/graphmind_support